# Informe Técnico:

## 1. Objetivo del proyecto

El objetivo principal es implementar y evaluar la eficacia de la arquitectura **MobileViT** (un Transformer de visión ligero) para la tarea de clasificación de imágenes utilizando el dataset `tf_flowers`. Se busca comparar el desempeño de modelos entrenados desde cero frente a estrategias de **fine-tuning** (ajuste fino), analizando cómo el procesamiento de datos impacta en la precisión final para dispositivos móviles.

## 2. Arquitectura general

La arquitectura MobileViT combina las ventajas de las **Redes Neuronales Convolucionales (CNN)** para capturar relaciones espaciales locales y los **Transformers** para modelar dependencias globales con baja latencia.

**Diagrama de flujo simplificado:** `Entrada (Imagen 256x256)` → `Convoluciones 3x3` → `Bloques Residuales Invertidos (MobileNetV2)` → `Bloques MobileViT` → `Global Average Pooling` → `Capa Lineal (Clasificación)`.

**Descripción de componentes clave:**

- **Bloques de Convolución:** Procesan la imagen inicialmente para reducir la resolución y capturar características locales.
- **Bloques MobileViT:** Son el núcleo del modelo. Realizan una convolución local, "desenrollan" (unfold) la imagen en parches, aplican un Transformer para obtener una representación global y luego "vuelven a enrollar" (fold) los parches en un mapa de características.
- **Separable Self-attention (en v2):** Reduce la complejidad computacional de O(k^2)$ a O(k), permitiendo una inferencia mucho más rápida en móviles.

## 3. Implementación técnica

La implementación se basa en el ecosistema de **TensorFlow y Keras**, utilizando modelos preentrenados de la librería de Hugging Face.

- **Herramientas:** TensorFlow 2.13+, Keras, Hugging Face `transformers`.

- **Dataset:** `tf_flowers`, que contiene 3,670 imágenes distribuidas en 5 clases (diente de león, margarita, tulipanes, girasoles y rosas).
- **Módulos de Preprocesamiento:**
  - **Escalado:** Redimensión de imágenes a 256×256 píxeles.
  - **Normalización:** Reescalado de píxeles (habitualmente a un rango de 0 a 1).
  - **Aumento de datos:** Uso de capas como `RandomFlip`, `RandomRotation` y `RandomZoom` para mejorar la robustez y evitar el sobreajuste (overfitting).

## 4. Evaluación (Métricas de desempeño)

El proyecto utiliza principalmente el **Accuracy** (exactitud) para medir el porcentaje total de valores correctamente clasificados.

| Modelo | Configuración | Val Accuracy |
|---|---|---|
| **MobileViT-XXS** | Entrenado desde cero | **79%** |
| **MobileViT-XS** | Fine-tuning (Dataset estándar) | **85%** |
| **MobileViT-XS** | Fine-tuning + Procesamiento avanzado | **96%** |

Otras métricas relevantes para este tipo de tareas incluyen la **Precisión** (calidad de positivos), **Recall** (capacidad de detección) y el **F1-Score**, que es especialmente útil si el dataset estuviera desbalanceado.

## 5. Resultados y ejemplos

- **Entrenamiento desde cero:** El modelo XXS logra un 79%, lo cual es notable para un modelo de solo ~1 millón de parámetros entrenado sin pesos previos.
- **Fine-tuning:** Al reutilizar pesos preentrenados en ImageNet-1k, el modelo XS sube al 85% rápidamente.
- **Optimización de Pipeline:** El salto al **96% de precisión** demuestra que un procesamiento de datos superior (mejor aumento de datos o técnicas de normalización específicas) es crítico para maximizar el rendimiento de los Transformers de visión.

## 6. Conclusiones y mejoras futuras

**Conclusiones:**

- MobileViT es una solución altamente eficiente para visión por computadora en dispositivos con recursos limitados.
- El **fine-tuning** supera significativamente al entrenamiento desde cero en datasets pequeños como `tf_flowers`.
- La combinación de convoluciones y transformers permite obtener precisiones de estado del arte (~96%) con una fracción de los parámetros de modelos tradicionales.

**Mejoras futuras:**

- **Cuantización:** Convertir el modelo a formato **TFLite** (entero de 8 bits) para reducir la huella de memoria y acelerar la inferencia en dispositivos móviles.
- **Poda (Pruning):** Eliminar pesos innecesarios para hacer el modelo aún más ligero sin perder precisión crítica.
- **Ajuste de Hiperparámetros:** Optimizar la tasa de aprendizaje y el tamaño de lote (batch size) para estabilizar la convergencia en el 4% restante de precisión.